Advanced Machine Learning Models

MFE5340: AI in FE: Quantitative Investment

Chuan Shi
School of Data Science

Spring 2026



香港中文大學（深圳）
The Chinese University of Hong Kong, Shenzhen



Table of Contents

1 Recap and Nonlinearity

- Recap and Nonlinearity
- Regression Trees
- Ensemble Models
- Neural Networks
- Deep Learning
- Economic Restrictions

- The essential problem in quantitative investment:

$$\begin{aligned} R^e_{i,t+1} &= \mathbb{E}[R^e_{i,t+1}|\boldsymbol{x}_{i,t}] + e_{i,t+1}, \\ \mathbb{E}[R^e_{i,t+1}|\boldsymbol{x}_{i,t}] &= f(\boldsymbol{x}_{i,t}). \end{aligned}$$

  This is a classic machine learning problem.

- Ways to learn about f:

Assumption	Econometrics	Machine Learning
$f(\boldsymbol{x}^i) = \text{avg}(y^j)_{\boldsymbol{x}^j \text{ near } \boldsymbol{x}^i}$	portfolio sort	regression trees, random forest, boosting
$f(\boldsymbol{x}^i)$ linear	OLS/GLS, Fama-MacBeth regression, regression based prediction	Penalized regression, e.g., ridge, Lasso (we have covered these!)
$f(\boldsymbol{x}^i)$ nonlinear	nonlinear least squares, GMM, MLE	neural networks, deep learning

Table adapted from Lasse Heje Pedersen's Bid Data Asset Pricing Lecture 5: Machine Learning in Asset Pricing.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models



Introducing Nonlinearity

- Traditional asset pricing models (e.g., CAPM, Fama-French) and quantitative investment strategies based on factor models assume **linear relationships** between asset returns and covariates.
- However, financial markets often exhibit **nonlinear behaviors** due to:
  - Complex investor psychology
  - Market shocks and extreme events
  - Asymmetric and threshold effects
- Nonlinearity is crucial for accurate predicting asset returns and risk management in quantitative investment.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

- **Polynomial Terms**
  - Higher-order powers of covariates (e.g., x^2, x^3) capture curvature in relationships.
  - Example: Barra has a nonlinear size factor (size^3) that captures the middle size effect on returns.

- **Interaction Terms**
  - Products of covariates (e.g., $x_1 \times x_2$) capture joint effects.
  - Example: See next few slides.

Empirical evidence suggests that interactions between covariates are the most important source of nonlinearity in predicting future returns.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models



Example: Polynomial Terms

- Consider the following predictive regression model from Nagel (2021):

$$R_{i,t+1} = \sum_{k=1}^{119} b_k R_{i,t-k} + \sum_{k=1}^{119} b_k R_{i,t-k}^2 + \sum_{k=1}^{119} d_k R_{i,t-k}^3 + e_{i,t+1},$$

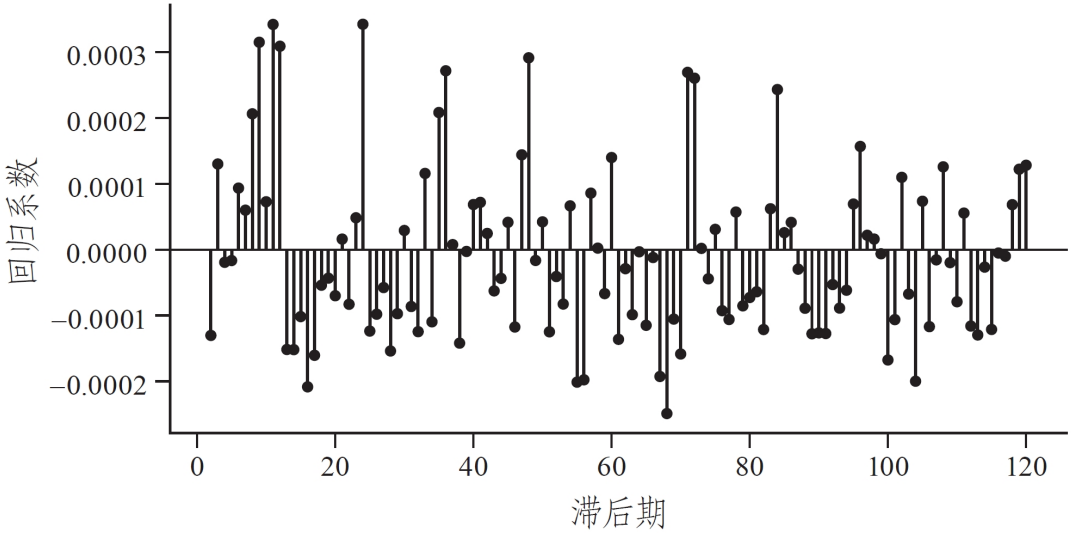
  where past returns, past returns squared, and past returns to the third power are used to predict $R_{i,t+1}$.

- Question for you: why are the first lag terms, i.e., $R_{i,t}$, $R_{i,t}^2$ and $R_{i,t}^3$, missing?

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

Example: Polynomial Terms

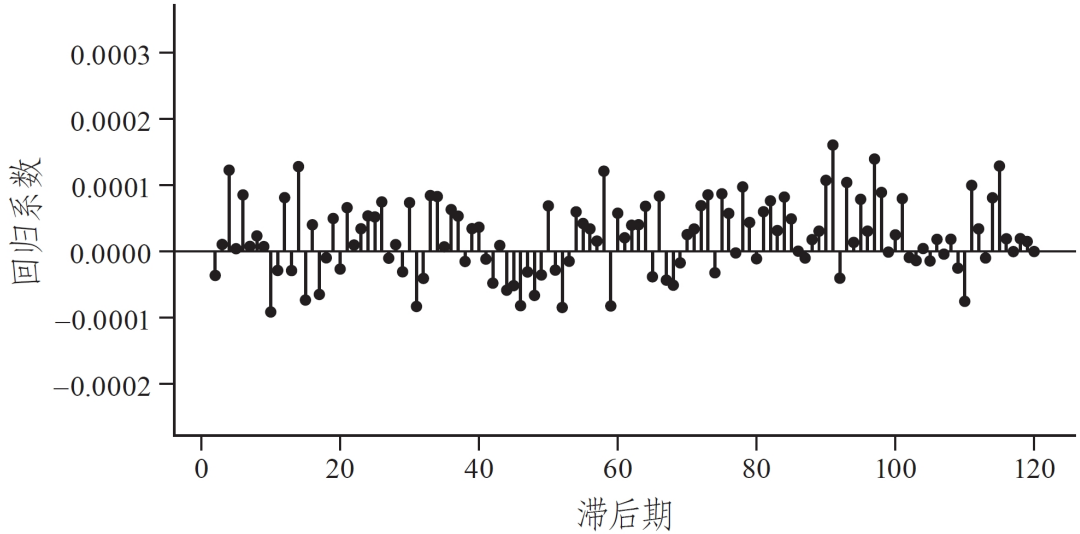
- Coefficients for past returns:



© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

Example: Polynomial Terms

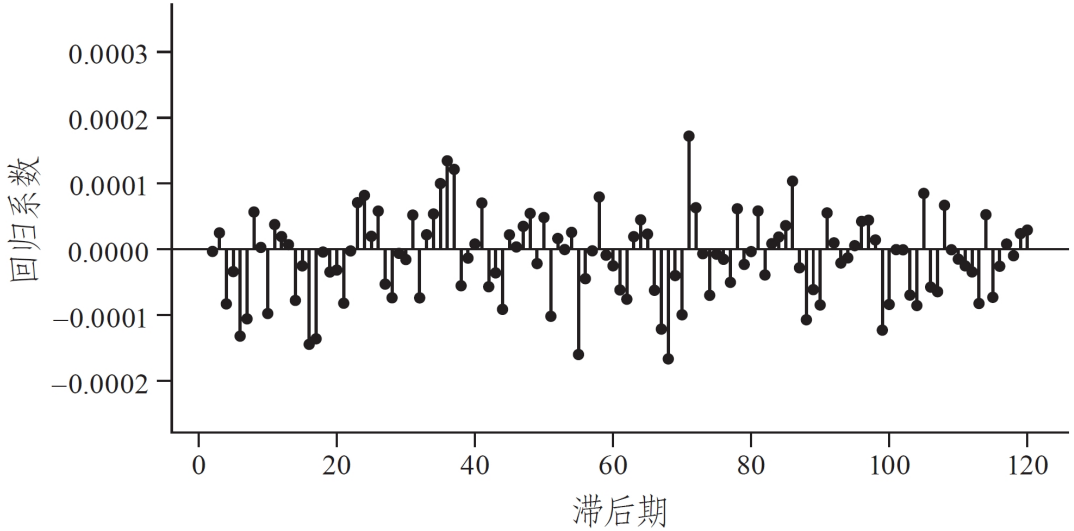
- Coefficients for squared past returns:



© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

Example: Polynomial Terms

- Coefficients for third power of past returns:



© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models



Why Interaction Matters

- **Market Behavior**
  - Investor sentiment and herding effects are inherently nonlinear.
  - Market reactions to shocks are often asymmetric.
- **Asset Returns**
  - Volatility clustering and leverage effects.
  - Tail risks and extreme events (e.g., crashes, bubbles).
- **Risk Factors**
  - Nonlinear relationships between asset prices and risk factors.
  - Interaction effects between multiple covariates.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models



Empirical Evidence: CGO and Low-Risk Anomalies

- Wang et al. (2017) combined capital gain overhang (CGO) with low-risk anomalies to explore how investors' attitudes towards risk and return vary depending on unrealized gains or losses.
- Financial theory suggests that higher risk should yield higher expected returns, but empirical data reveals an inverse relationship: stocks with higher risk tend to have lower returns, leading to low-risk anomalies such as IVOL and BAB.
- In a state of unrealized losses, investors tend to seek high risk stocks, hoping to recover losses through volatile assets, leading to further overvaluation of these stocks. Conversely, in a state of unrealized gains, investors become more risk-averse, often selling high risk stocks, which results in their undervaluation.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

Empirical Evidence: CGO and Low-Risk Anomalies

- CGO1 represents the group with the largest unrealized losses, and CGO5 represents the group with the highest unrealized gains. P1 is the group with the lowest risk and P5 is the group with the highest risk.

Portfolio	CGO1	CGO3	CGO5	Diff-in-Diff	CGO1	CGO3	CGO5	Diff-in-Diff
	Proxy = β				Proxy = RETVOL			
P1	0.596	0.494	0.545		0.679	0.480	0.584	
P3	0.504	0.537	0.781		0.429	0.531	0.810	
P5	−0.046	0.378	0.885		−0.383	0.417	1.184	
P5 - P1	−0.642	−0.116	0.340	0.983	−1.062	−0.063	0.601	1.663
t-stat	(−2.31)	(−0.49)	(1.52)	(3.92)	(−2.84)	(−0.21)	(2.19)	(4.52)
FF3-α	−0.930	−0.429	0.172	1.103	−1.253	−0.408	0.334	1.587
t-stat	(−4.28)	(−2.59)	(0.97)	(4.24)	(−4.41)	(−1.76)	(1.43)	(4.20)
	Proxy = IVOL				Proxy = CFVOL			
P1	0.875	0.477	0.669		0.742	0.598	0.699	
P3	0.233	0.492	0.797		0.485	0.412	0.843	
P5	−1.050	0.072	0.989		0.274	0.306	0.936	
P5 - P1	−1.924	−0.405	0.320	2.244	−0.469	−0.292	0.237	0.706
t-stat	(−6.00)	(−1.62)	(1.58)	(7.97)	(−2.00)	(−1.48)	(1.25)	(2.68)
FF3-α	−2.093	−0.678	0.132	2.225	−0.515	−0.345	0.143	0.658
t-stat	(−8.48)	(−3.59)	(0.73)	(8.10)	(−2.47)	(−2.01)	(0.84)	(2.41)

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

Empirical Evidence: IVOL and Valuation

	Highest IVOL	Next 20%	Next 20%	Next 20%	Lowest IVOL	Highest –Lowest	All Stocks
Most Overpriced	−1.89	−0.95	−0.72	−0.47	−0.39	−1.50	−0.81
(Top 20%)	(−12.05)	(−7.39)	(−4.90)	(−3.62)	(−3.04)	(−7.36)	(−8.14)
Next 20%	−0.88	−0.41	−0.31	−0.21	−0.04	−0.84	−0.23
	(−5.86)	(−3.36)	(−3.00)	(−2.08)	(−0.44)	(−4.41)	(−3.88)
Next 20%	−0.09	−0.01	−0.05	−0.12	0.02	−0.10	−0.07
	(−0.53)	(−0.09)	(−0.48)	(−1.29)	(0.18)	(−0.53)	(−1.47)
Next 20%	−0.15	0.07	0.17	0.18	0.23	−0.38	0.18
	(−0.80)	(0.63)	(1.87)	(2.33)	(3.22)	(−1.78)	(4.45)
Most Underpriced	0.56	0.68	0.51	0.33	0.14	0.41	0.28
(Bottom 20%)	(3.27)	(4.91)	(5.02)	(4.10)	(2.04)	(2.16)	(5.67)
Most Overpriced –	−.44	−1.63	−1.23	−0.81	−0.53	−1.91	−1.09
Most Underpriced	(−11.07)	(−8.65)	(−6.43)	(−5.02)	(−3.43)	(−7.62)	(−8.05)
All Stocks	−0.69	−0.12	−0.00	0.05	0.08	−0.78	
	(−6.09)	(−1.56)	(−0.01)	(1.07)	(1.86)	(−5.50)	

Source: Stambaugh, Yu and Yuan (2015)

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models



Table of Contents

2 Regression Trees

- Recap and Nonlinearity
- Regression Trees
- Ensemble Models
- Neural Networks
- Deep Learning
- Economic Restrictions

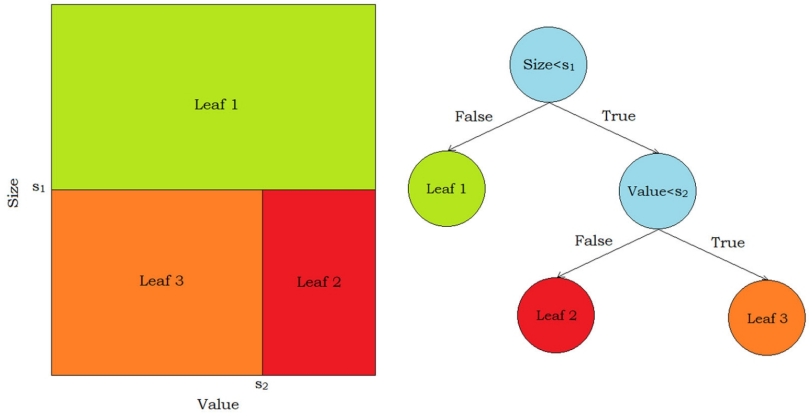
© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

- **Definition**
  - Regression trees are a nonparametric machine learning technique used to model decisions based on input variables.
- **Structure**
  - They form a tree-like structure where each decision (or split) is made based on specific criteria.
- **Application**
  - Useful for predicting outcomes influenced by multiple interacting variables, especially when relationships are nonlinear.
  - E.g., In quantitative investment, regression trees can be used to sort stocks by specific characteristics. At each node, the tree assesses whether a cut-off at a specific value of the chosen variable can divide stocks into two groups with similar characteristics.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

Example 1: Market Capitalization

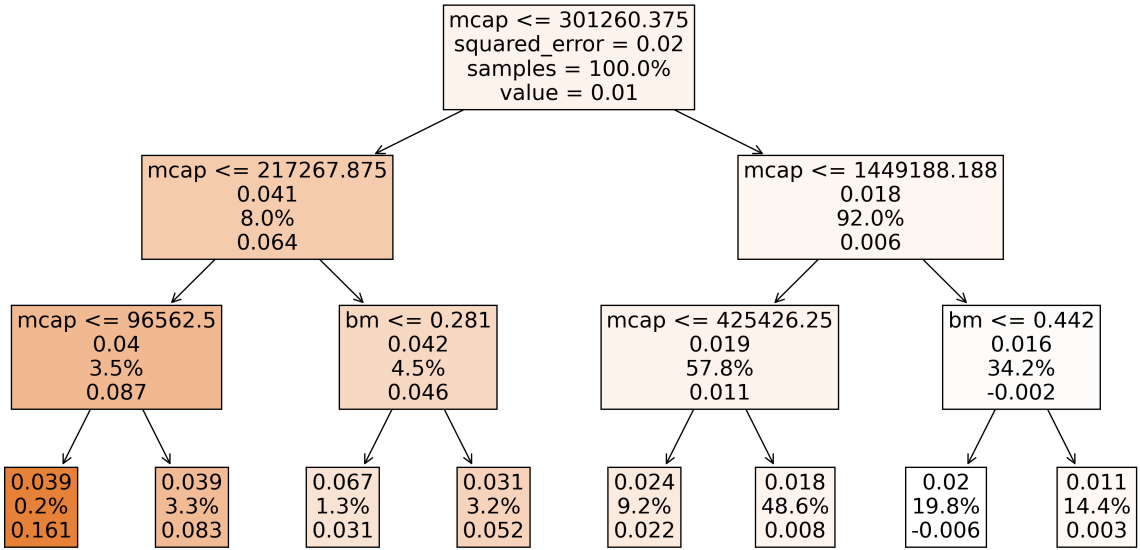
- The tree might evaluate sorting stocks by market capitalization.
- It applies a cut-off at a particular value to create two portfolios.
- This helps in identifying portfolios with similar market capitalization traits, aiding in better asset pricing and investment decisions.



Source: Bagnara (2024)

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

Example 2: Size and Value Interaction



© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

- A regression tree is built by selecting splits that minimize prediction error. At each node, the dataset is divided into two subsets that minimize the mean squared error (MSE) of predicted returns:

$$\mathcal{L}(C, C_{\text{left}}, C_{\text{right}}) = \frac{1}{N_{\text{left}}} \sum (R_{i,t+1} - \theta_{\text{left}})^2 + \frac{1}{N_{\text{right}}} \sum (R_{i,t+1} - \theta_{\text{right}})^2,$$

  where C denotes the dataset from the preceding step, while the new subsets are C_{left} and C_{right}. N_{left} and N_{right} are the corresponding number of observations.

- The predicted return is the average return of all stocks within that group, i.e.,

$$\theta_{\text{left}} = \frac{1}{N_{\text{left}}} \sum_{z_{i,t} \in C_{\text{left}}} R_{i,t+1}; \quad \theta_{\text{left}} = \frac{1}{N_{\text{right}}} \sum_{z_{i,t} \in C_{\text{right}}} R_{i,t+1}.$$

Note: This slide incorporates material from Professor Doron Avramov's lecture notes, used with permission.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

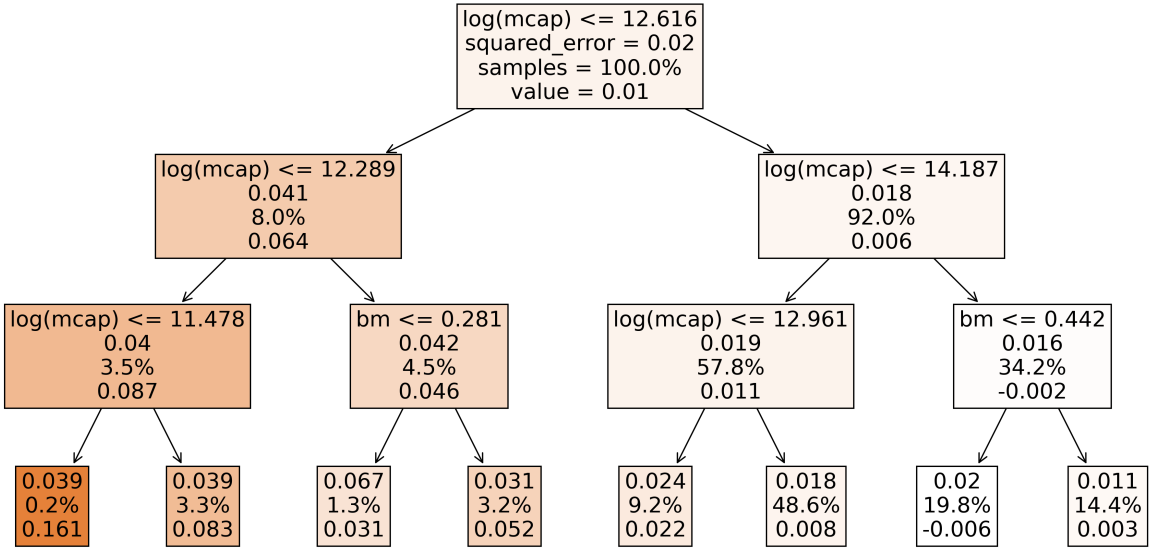


Pros: Handling Data Complexity

- **Nonlinearity & Interactions**
  - Captures nonlinear relationships without manual transformations.
  - Models variable interactions inherently.
- **Distribution-Free Flexibility**
  - No assumptions on linearity, normality, or homoscedasticity.
  - Adapts to outliers and irregular patterns.
- **Missing Data Handling**
  - Splits data using available features, avoiding imputation.
  - Maximizes data utilization.
- **Outlier Resilience**
  - Less sensitive to outliers than linear models.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

Example 2 Revisited: Log(Size) and Value Interaction



© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models



Pros: Simplicity and Transparency

- **Simplicity & Transparency**
  - Clear decision rules visualized as a tree structure.
  - Easy to explain to non-technical audiences.
- **Variable Importance**
  - Automatically ranks predictors by their impact on reducing prediction error.
  - Provides actionable insights for feature selection.
- **Scalability**
  - Efficient training on large datasets.
  - Parallelizable for high-performance computing, e.g., Random Forests (next!).

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models



Limitations: Overfitting

- **Overfitting**
  - **Trees are among the prediction methods most prone to overfit, and therefore must be regularized.**
  - Deep trees with excessive nodes memorize training data, failing to generalize to out-of-sample periods.
- **Pruning Trade-offs**
  - Costly cross-validation required to balance complexity and generalizability.
  - Aggressive pruning may discard economically meaningful splits.
- **Model Instability & Data Sensitivity**
  - Minor perturbations in training data lead to structurally different trees.
  - Problematic for asset pricing models requiring stable factor definitions.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models



Limitations: Bias, Smoothness and Suboptimality

- **Feature Selection Bias**
  - Variables with more unique values (e.g., market cap, turnover) dominate splits, even if economically irrelevant.
  - In asset pricing, this may overweight noisy microstructural variables over fundamental factors. Solution: Pre-screen features via economic theory.

- **Smoothness Issues**
  - Single trees struggle to capture smooth nonlinearities.
  - Step-function predictions create jagged portfolio sorts, introducing turnover costs.

- **Greedy Optimization**
  - Myopic splits maximize immediate error reduction but ignore long-term interactions.
  - Suboptimal for multi-period portfolio optimization.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models



Table of Contents

3 Ensemble Models

- Recap and Nonlinearity
- Regression Trees
- Ensemble Models
- Neural Networks
- Deep Learning
- Economic Restrictions

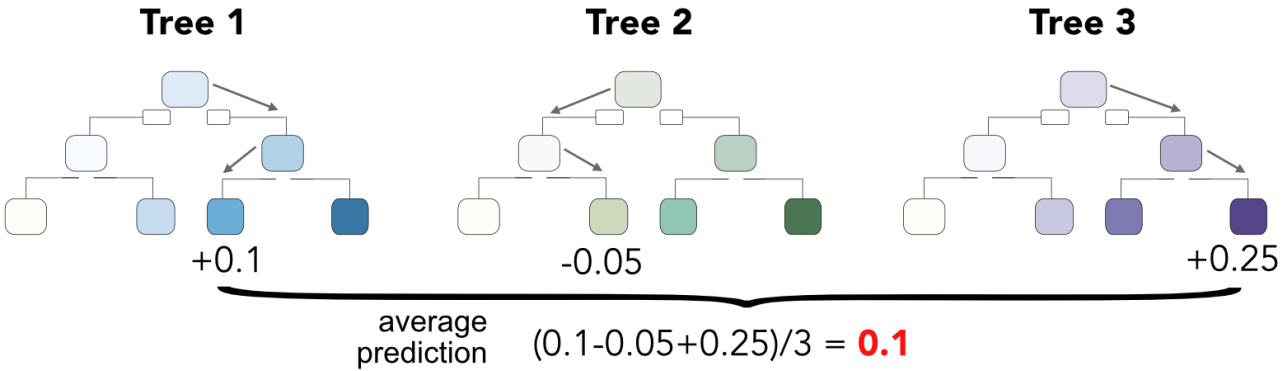
© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

- **Ensemble Learning**
  - A random forest combines multiple decision trees to enhance predictive accuracy and mitigate overfitting.
  - It is superior to single trees by exploiting the "wisdom of crowds" principle.

- **Key Innovations**
  - *Bootstrap Aggregating (Bagging)*: Trains each tree on a random subset of data (with replacement).
  - *Feature Randomization*: At each split, only a random subset of features is evaluated, decorrelating trees.

- **Averaging Predictions**
  - In the prediction phase, the random forest aggregates outputs from its constituent trees: for regression tasks, the final prediction is the arithmetic mean of all tree predictions, while for classification tasks, majority voting is used to determine the final output.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models



- Source: Coqueret and Guida (2023)

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

- **Variance Reduction**
  - Random forests improve prediction performance by reducing variance.
  - Individual trees may suffer from high variance and overfit the training data, but by averaging the predictions of multiple trees, the variance is reduced, resulting in more accurate and stable predictions.

- **Guard Against Overfitting**
  - Breiman (2001) showed that as the number of trees grows to infinity, the inaccuracy converges to some finite number. This explains why random forests are not prone to overfitting.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models



Random Forest: Illustration

- **Data Preparation**
  - Select 20 features, align the features with stock returns, and preprocess the data by shifting the returns for prediction.
- **Stock Pool**
  - CSI500 index components; monthly frequency.
- **Data Splitting**
  - Split data into training and testing sets using Sept 30, 2022 as the split date. Training data before this date and testing data after.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models



Random Forest: Illustration

- **Model Training**
  - Decision Tree Regressor (max depth = 10).
  - Random Forest Regressor (100 trees, max depth = 10, max features = 4).
  - **Note:** We did not fine-tune the hyperparameters using cross-validation.

- **Model Evaluation** (Mean Squared Error on the test set)
  - Decision Tree MSE: 0.009721
  - Random Forest MSE: 0.009119

- **Summary**
  - Both models show similar performance.
  - Random Forest has a slight edge in predictive accuracy.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models



Boosted Trees: Adaboost

- AdaBoost (Adaptive Boosting) is a powerful ensemble method typically used for *classification* tasks (`AdaBoost.M1`).
- AdaBoost can also be adapted for *regression* problems, referred to as `AdaBoost.R2`.
- Goal: Combine weak regressors to form a strong predictor for continuous output.
- Works by training weak learners sequentially, with each learner focusing more on the previously mispredicted data points.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

Schematic of AdaBoost

- AdaBoost aims to improve prediction accuracy by focusing on difficult-to-predict samples. Each new model in the sequence tries to correct the mistakes made by the previous models.
- Source: Hastie, Tibshirani and Friedman (2009)

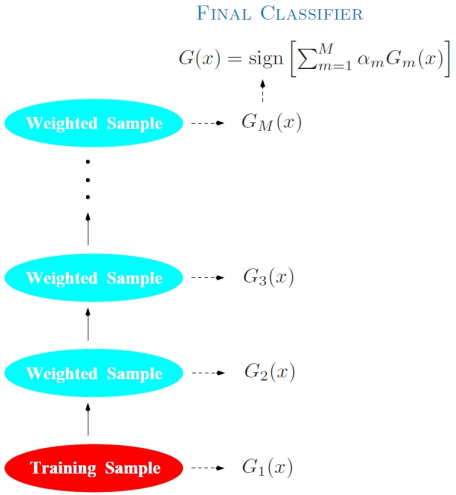


FIGURE 10.1. *Schematic of AdaBoost. Classifiers are trained on weighted versions of the dataset, and then combined to produce a final prediction.*

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

Algorithm 10.1 *AdaBoost.M1.*

1. Initialize the observation weights $w_i = 1/N,\ i = 1, 2, \ldots, N$.
2. For $m = 1$ to M:
   (a) Fit a classifier $G_m(x)$ to the training data using weights w_i.
   (b) Compute
   $$\text{err}_m = \frac{\sum_{i=1}^{N} w_i I(y_i \neq G_m(x_i))}{\sum_{i=1}^{N} w_i}.$$
   (c) Compute $\alpha_m = \log((1 - \text{err}_m)/\text{err}_m)$.
   (d) Set $w_i \leftarrow w_i \cdot \exp[\alpha_m \cdot I(y_i \neq G_m(x_i))],\ i = 1, 2, \ldots, N$.
3. Output $G(x) = \text{sign}\left[\sum_{m=1}^{M} \alpha_m G_m(x)\right]$.

- Source: Hastie, Tibshirani and Friedman (2009)

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

1. Initialize the weights $w_i = 1/N, i = 1, 2, \cdots, N$.

2. For $m = 1$ to M:
   2.1 Train a weak model $f_m(\boldsymbol{x})$ to the training data using w_i.
   2.2 For each observation $i = 1, \cdots, N$, calculate the error:

$$e_i = \frac{|y_i - f_m(\boldsymbol{x}_i)|}{S_m},$$

   where $S_m = \max_i\{|y_i - f_m(\boldsymbol{x}_i)|\}$.
   2.3 Calculate error of the model $\text{err}_m = \sum_{i=1}^{N} w_i e_i$. If $\text{err}_m \geq 0.5$, end iteration and set M to $m - 1$.
   2.4 Compute $\alpha_m = \text{err}_m/(1 - \text{err}_m)$. (The definition here is different from the one in `AdaBoost.M1`.)
   2.5 Set $w_i \leftarrow w_i \alpha_m^{1-e_i}$ (and normalize it).

3. Set the final fitted value to the **weighted median** of $f_m(\boldsymbol{x}_i)$ using weight $\log(1/\alpha_m)$ for model m.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

Adaboost: Illustration

- **Model Training**
  - Decision Tree Regressor (max depth = 10).
  - Adaboost Regressor (max estimator = 100, learning rate = 0.1).
  - **Note:** We did not fine-tune the hyperparameters using cross-validation.
- **Model Evaluation** (Mean Squared Error on the test set)
  - Decision Tree MSE: 0.009721
  - Adaboost MSE: 0.009355
- **Summary**
  - Both models show similar performance.
  - Adaboost has a slight edge in predictive accuracy.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

Gradient Boosted Regression Trees

- Gradient Boosting Regression Trees (GBRT) is another powerful ensemble method commonly used for regression.
- GBRT builds an ensemble of weak regressors, typically shallow trees, trained sequentially to **minimize the residual errors** of previous models.
- Goal: Combine weak regressors to create a strong predictor by fitting trees to the residuals of the previous model.
- GBRT works by updating the model in a *gradient descent* fashion, where each weak model is trained to minimize the loss of the previous iteration.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models



GBRT Algorithm

1. Initialize the model with a constant value f_0 (typically the mean of the target y_i): $f_0 = \frac{1}{N}\sum_{i=1}^{N} y_i$.
2. For $m = 1$ to M:
   - 2.1 Compute the residuals $r_i = y_i - f_{m-1}(\boldsymbol{x}_i)$.
   - 2.2 Fit a weak learner $h_m(\boldsymbol{x})$ (a shallow decision tree) to the residuals r_i, minimizing the loss (mean squared error):
   $$h_m(\boldsymbol{x}) = \arg\min_{h} \sum_{i=1}^{N} \left(r_i - h(\boldsymbol{x}_i)\right)^2 .$$
   - 2.3 Calculate the optimal step size γ_m s.t.
   $$\gamma_m = \arg\min_{\gamma} \sum_{i=1}^{N} \left(y_i - f_{m-1}(\boldsymbol{x}_i) - \gamma h_m(\boldsymbol{x}_i)\right)^2 .$$
   - 2.4 Update the model with the new learner $f_m(\boldsymbol{x}) = f_{m-1}(\boldsymbol{x}) + \gamma_m h_m(\boldsymbol{x})$.
3. Set the final prediction to the output of the ensemble model $f_M(\boldsymbol{x})$ after M iterations.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

- GBRT is an **additive model**, where each new model is added to the existing ensemble to improve predictions, i.e., $f_m(\boldsymbol{x}) = f_{m-1}(\boldsymbol{x}) + \gamma_m h_m(\boldsymbol{x})$.

- The final model is the sum of all the weak learners added sequentially:

$$f_M(\boldsymbol{x}) = f_0(\boldsymbol{x}) + \sum_{m=1}^{M} \gamma_m h_m(\boldsymbol{x}),$$

  where each tree $h_m(\boldsymbol{x})$ contributes to reducing the error.

- This sequential improvement approach allows GBRT to model complex, non-linear relationships by combining simple weak learners.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

- GBRT can be understood as an application of **gradient descent** to decision trees. Each iteration tries to reduce the mean squared error by moving in the direction of the negative gradient.

- The residuals $r_i = y_i - f_{m-1}(\boldsymbol{x}_i)$ are effectively the negative gradient of the MSE, i.e.,

$$r_i \propto -\frac{\partial \mathcal{L}(f_{m-1})}{\partial f_{m-1}(\boldsymbol{x}_i)}$$

  with the MSE at iteration $m-1$ given by

$$\mathcal{L}(f_{m-1}) = \frac{1}{N}\sum_{i=1}^{N}(y_i - f_{m-1}(\boldsymbol{x}_i))^2.$$

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

- Regular gradient descent directly updates the model by:

$$\begin{aligned} f_m(\boldsymbol{x}_i) &= f_{m-1}(\boldsymbol{x}_i) - \gamma \frac{\partial \mathcal{L}(f_{m-1})}{\partial f_{m-1}(\boldsymbol{x}_i)} \\ &= f_{m-1}(\boldsymbol{x}_i) + \gamma(y_i - f_{m-1}(\boldsymbol{x}_i)), \end{aligned}$$

  whereas GBRT fits a new weak learner to the residuals r_i and update the model by:

$$f_m(\boldsymbol{x}_i) = f_{m-1}(\boldsymbol{x}_i) + \gamma h_m(\boldsymbol{x}_i),$$

- The step size γ_m controls how much we move in the direction of the negative gradient. It determines the size of the adjustment to the model, allowing us to make gradual improvements and avoid overfitting.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models



Gradient Descent vs. GBRT

Gradient Descent	GBRT
Directly optimizes model parameters (e.g., weights)	Optimizes predictions step-by-step by adding new models (trees)
Suitable for parametric models (e.g., neural networks)	Suitable for non-parametric models (tree-based models)
The update direction is the negative gradient	The negative gradient direction is fitted using a regression tree

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models



Properties of GBRT

- High accuracy and flexibility.
- Can handle complex relationships in data (non-linear).
- Robust to outliers and missing data.
- Can minimize any loss function (e.g., mean squared error, mean absolute error) depending on the task at hand.
- The learning rate and tree depth provide regularization, preventing overfitting by controlling the contribution of each tree and limiting the complexity of each tree.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models



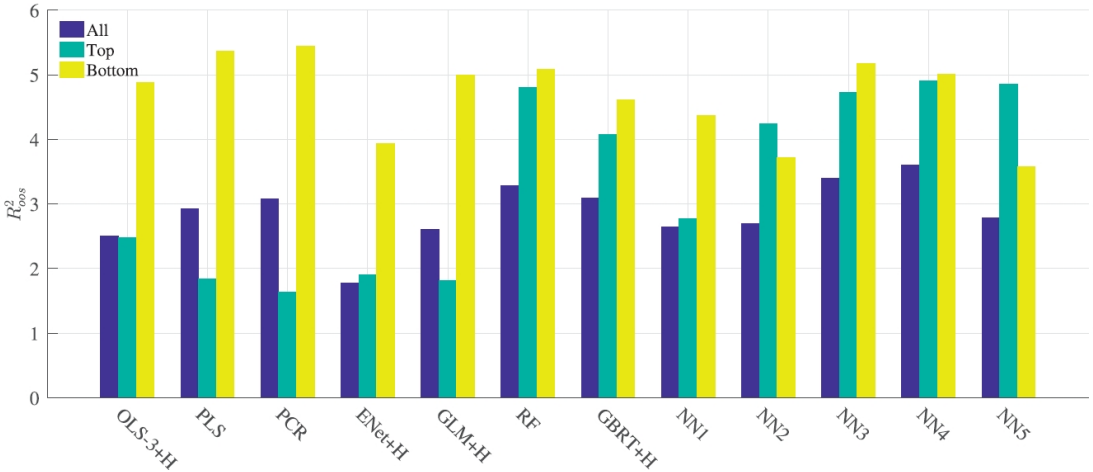
GBRT: Illustration

- **Model Training**
  - Decision Tree Regressor (max depth = 10).
  - GBRT Regressor (max estimator = 100, max depth = 3, learning rate = 0.1).
  - **Note:** We did not fine-tune the hyperparameters using cross-validation.

- **Model Evaluation** (Mean Squared Error on the test set)
  - Decision Tree MSE: 0.009721
  - GBRT MSE: 0.009244

- **Summary**
  - Both models show similar performance.
  - GBRT has a slight edge in predictive accuracy.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models



Annual return forecasting R^2_{oos} (see the legend to Table 1).

- Source: Gu, Kelly and Xiu (2020)

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models



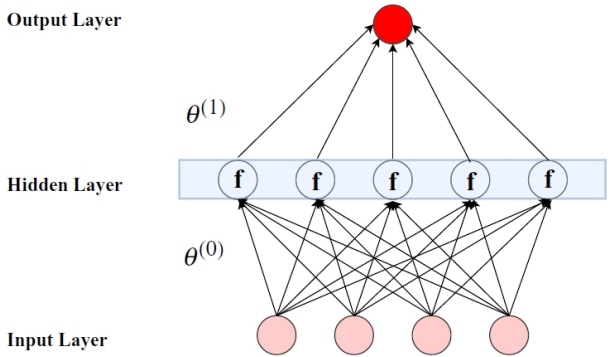
Table of Contents

4 Neural Networks

- Recap and Nonlinearity
- Regression Trees
- Ensemble Models
- Neural Networks
- Deep Learning
- Economic Restrictions

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

- Neural networks (NNs), modeled after the structure of the human brain, are composed of layers of "neurons" interconnected by "synapses" that carry signals between the layers.
- The network handles information by passing it from an input layer, through one or more hidden layers, to an output layer, which is commonly referred to as a Feed-Forward Network (FFN).
- The output layer makes predictions similar to fitted values in regression analysis.



Source: Gu, Kelly and Xiu (2020)

Note: This slide incorporates material from Professor Doron Avramov's lecture notes, used with permission.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

- Input layer: each neuron k is just the kth feature, i.e., $a_k^{(0)} = x_k$.
- Hidden layer $h = 1, \cdots, H$: each neuron x_j^h is a (non)linear *activation* function g of prior neurons:

$$a_j^{(h)} = g\left(\theta_{j,0}^{(h-1)} + \sum_i \theta_{j,i}^{(h-1)} a_i^{(h-1)}\right).$$

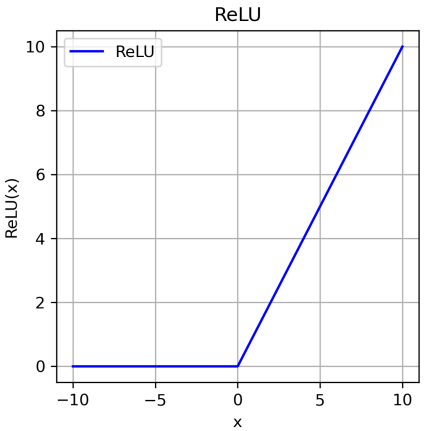
- Output layer (i.e., the prediction):

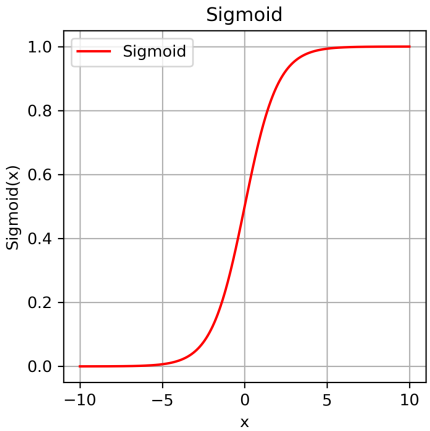
$$\hat{f}(x) = \theta_0^{(H)} + \sum_i \theta_i^{(H)} a_i^{(H)}.$$

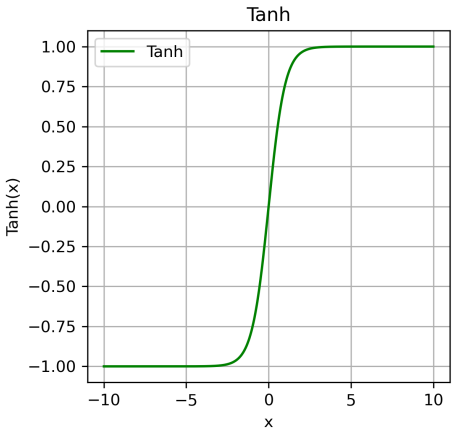
- At each neuron, we first compute a linear function of past neurons (with weights $\boldsymbol{\theta}$), and then transform it with the activation function g.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

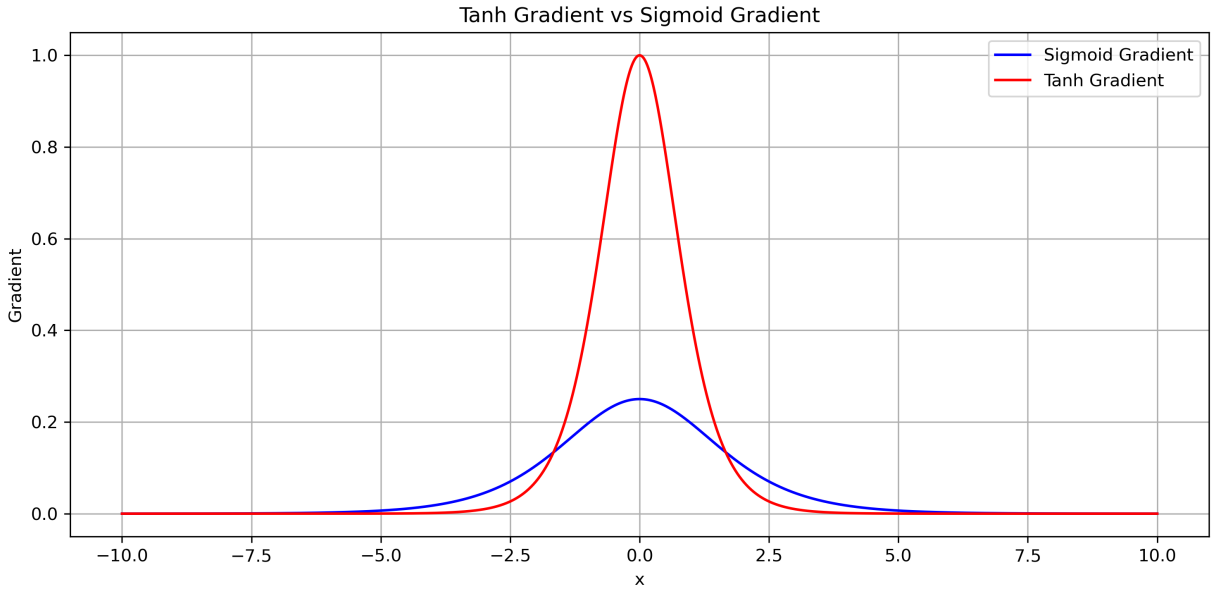
$\text{ReLU}(x) = \max(0, x)$ Sigmoid: $s(x) = \dfrac{1}{1 + e^{-x}}$ tanh: $2s(2x) - 1$







© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models



© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

- Taking ReLU as an example, the neural network can be written as:

$$\text{output} = \max\left(\max\left(\max\left(\mathbf{A}^{(0)}\boldsymbol{\Theta}^{(0)}, 0\right)\boldsymbol{\Theta}^{(1)}, 0\right)\cdots\boldsymbol{\Theta}^{(H-1)}, 0\right)\boldsymbol{\Theta}^{(H)}.$$

  Yes, there are a lot of max!!!

- We run an optimization to minimize the loss function. For continuous predicted variable, mean squared errors (MSE) is the default choice. (But is this always a good choice for finance?)

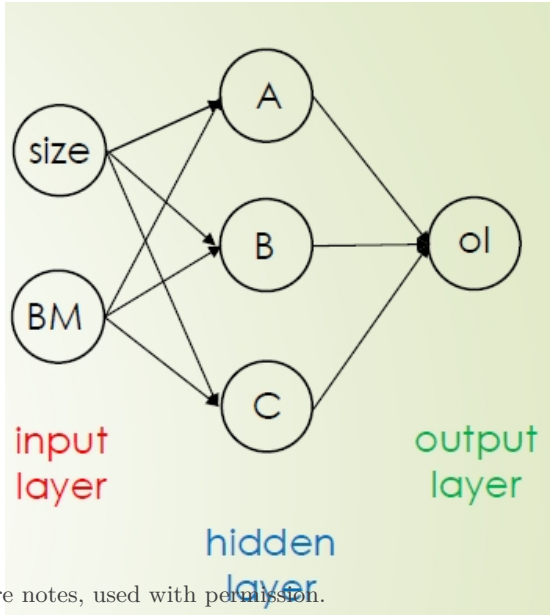
- If the activation function is linear – simply ignore the MAX operator in the above equation. **NN boils down to OLS.**

Note: This slide incorporates material from Professor Doron Avramov's lecture notes, used with permission.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

An Example

- $\text{input}_A^{(1)} = g\left(\text{size} \times \theta_{A,\text{size}}^{(0)} + \text{BM} \times \theta_{A,\text{BM}}^{(0)} + \theta_{A,0}^{(0)}\right)$
- $\text{input}_B^{(1)} = g\left(\text{size} \times \theta_{B,\text{size}}^{(0)} + \text{BM} \times \theta_{B,\text{BM}}^{(0)} + \theta_{B,0}^{(0)}\right)$
- $\text{input}_C^{(1)} = g\left(\text{size} \times \theta_{C,\text{size}}^{(0)} + \text{BM} \times \theta_{C,\text{BM}}^{(0)} + \theta_{C,0}^{(0)}\right)$
- $\text{output} = a_A^{(1)} \times \theta_A^{(1)} + a_B^{(1)} \times \theta_B^{(1)} + a_C^{(1)} \times \theta_C^{(1)} + \theta_0^{(1)}$
- To find the parameters, you minimize the sum of squared errors (or maximize IC) by aggregating over all stocks and all months.



Note: This slide incorporates material from Professor Doron Avramov's lecture notes, used with permission.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

- Hyperparameters: number of hidden layers/number of neurons per layer, regularizaiton parameters, learning rate.
- Training the network:
  - Training a neural network involves minimizing a loss function through gradient descent, an optimization technique that updates the network's weights iteratively:

$$\text{new weight} = \text{old weight} - (\text{learning rate} \times \text{gradient of the loss w.r.t the weight}).$$

- The learning rate is a crucial hyperparameter that controls the step size.
  - If it is too large, the network may overshoot the minimum.
  - If it is too small, training can be slow or get stuck in local minima.

Note: This slide incorporates material from Professor Doron Avramov's lecture notes, used with permission.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

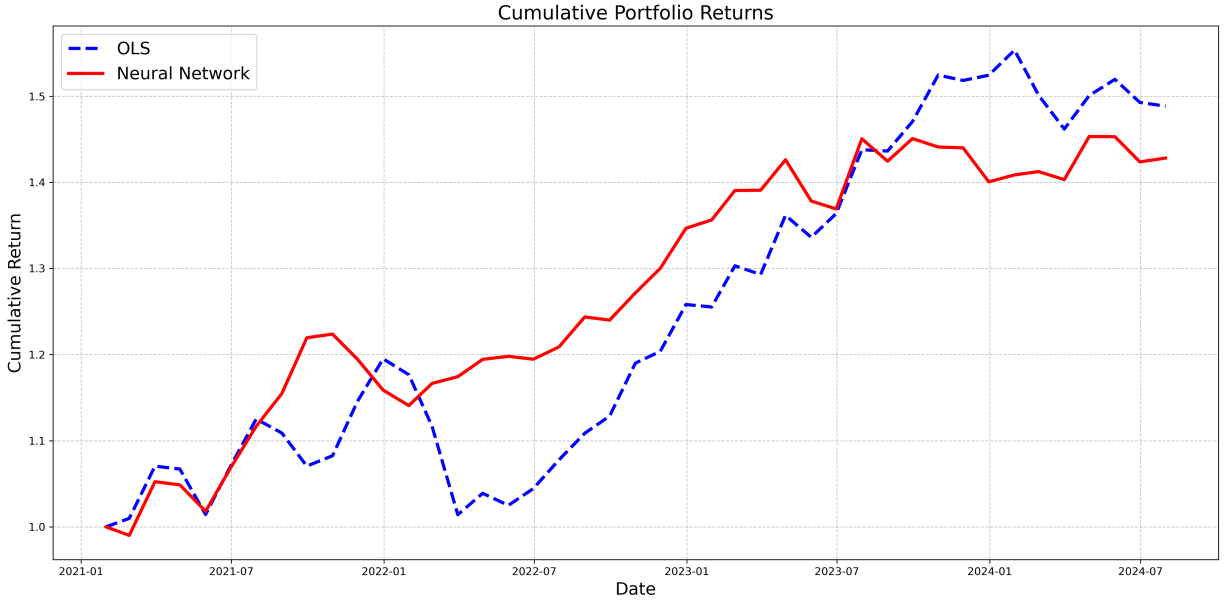
- Backpropagation is the fundamental algorithm used for training neural networks, enabling the computation of gradients needed to adjust the weights.

- The algorithm operates by utilizing the chain rule to calculate the gradient of the loss with respect to each weight, beginning at the output layer and moving backward through the network.

- Despite its effectiveness, backpropagation faces several challenges:
  - Non-convexity: The loss function often contains numerous local minima and saddle points, complicating the search for the global minimum.
  - Vanishing gradients: In deep neural networks, gradients can diminish significantly as they propagate backward, leading to slower convergence during training.

Note: This slide incorporates material from Professor Doron Avramov's lecture notes, used with permission.

© 2025 – 2026 Chuan Shi



© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

- Autoencoders are a type of *unsupervised* learning model that perform dimensionality reduction by compressing input data into a lower-dimensional representation, then reconstructing the original input from this compressed version.
- Useful for extracting latent features from complex data, e.g., asset pricing applications.
- Training autoencoders involves the minimization error from reconstructing its inputs $\boldsymbol{x}$ in two steps:

$$\boldsymbol{x} \xrightarrow{f(\boldsymbol{x})} \boldsymbol{z} \xrightarrow{g(\boldsymbol{z})} \hat{\boldsymbol{x}}$$

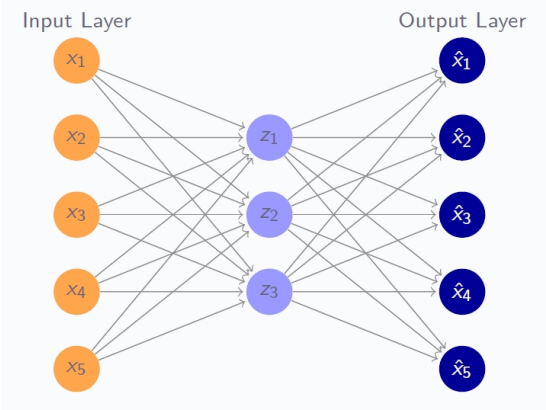
  - Encoding $\boldsymbol{x} \xrightarrow{f(\boldsymbol{x})} \boldsymbol{z}$: Compress input data into a lower-dimensional representation.
  - Decoding $\boldsymbol{z} \xrightarrow{g(\boldsymbol{z})} \hat{\boldsymbol{x}}$: Reconstruct the original data from the compressed representation.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

Autoencoders as Neural Networks

- **Input Layer (orange):** Represents the original input variables.
- **Hidden Layer (purple):** Encodes input data into a compressed representation.
- **Output Layer (blue):** Reconstructs the input data from the compressed representation.

- Orange → Purple: Encoding process.
- Purple → Blue: Decoding process.



© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

- Given the choice of activation functions, f and g, and encoding dimension, K, train an autoencoder to minimize the reconstruction error:

$$\frac{1}{NT}\|\mathbf{X} - \hat{\mathbf{X}}\|_F^2$$

  where $\|\cdot\|_F$ is the Frobenius norm.

- Add regularization terms to penalize weights.

- Increase network depth by introducing additional hidden layers around the encoding layer.

- **Does minimizing $\|\mathbf{X} - \hat{\mathbf{X}}\|_F^2$ remind you of PCA?** Let's compare autoencoder with PCA.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models



Comparison with PCA

- PCA: A linear technique that reduces dimensions by projecting data onto orthogonal axes, retaining as much variance as possible.
- Autoencoders: Use nonlinear neural networks to capture more complex patterns in the data, and minimize reconstruction error via a loss function (e.g., MSE).
- Autoencoders use activation functions like ReLU to handle nonlinear transformations, making them more versatile than PCA.
  - For example, in financial data where relationships between covariates are often nonlinear, autoencoders can uncover hidden factors that explain more variance than traditional linear methods.
- What if we consider a linear autoencoder? (i.e., the activation function is linear.) Recall that PCA is the best low-rank approximation from the PCA lecture.

Note: This slide incorporates material from Professor Doron Avramov's lecture notes, used with permission.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

- Linear Autoencoder:

$$\min_{\mathbf{W}_1 \in \mathbb{R}^{N \times K}, \mathbf{W}_2 \in \mathbb{R}^{K \times N}} \|\mathbf{X} - \mathbf{X}\mathbf{W}_1\mathbf{W}_2\|_F^2.$$

  - $\mathbf{W}_1$: Encoder matrix for dimensionality reduction.
  - $\mathbf{W}_2$: Decoder matrix for reconstruction.
  - No orthogonality or unit norm constraints on $\mathbf{W}_1$ and $\mathbf{W}_2$.

- PCA:

$$\min_{\mathbf{W} \in \mathbb{R}^{N \times K}} \|\mathbf{X} - \mathbf{X}\mathbf{W}\mathbf{W}^\top\|_F^2, \quad \text{s.t. } \mathbf{W}^\top\mathbf{W} = \mathbf{I}.$$

  - $\mathbf{W}$: Projects high-dimensional data to a lower-dimensional subspace.
  - $\mathbf{W}^\top$: Reconstructs the data back to the original space.
  - Orthogonality constraint ensures unit norm and minimum reconstruction error.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

Linear Autoencoder vs. PCA

Feature	PCA	Autoencoder
Projection Matrix	Orthogonal ($\mathbf{W}$)	Learned ($\mathbf{W}_1, \mathbf{W}_2$)
Optimization	Closed-form (via SVD)	Gradient-based optimization
Nested Solution	Yes	No. Must train each K separately

- Autoencoders offer greater flexibility and can uncover more complex features by incorporating non-linear transformations.
- They allow the use of diverse loss functions beyond MSE.
- Regularization techniques can be applied to mitigate overfitting, whereas PCA might mistakenly interpret noise as meaningful latent components, which could hinder generalization out-of-sample.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

- **Traditional Factor Models:**
  - Assume *static* β – time-invariant factor exposures.
  - Limited ability to capture dynamic market conditions.

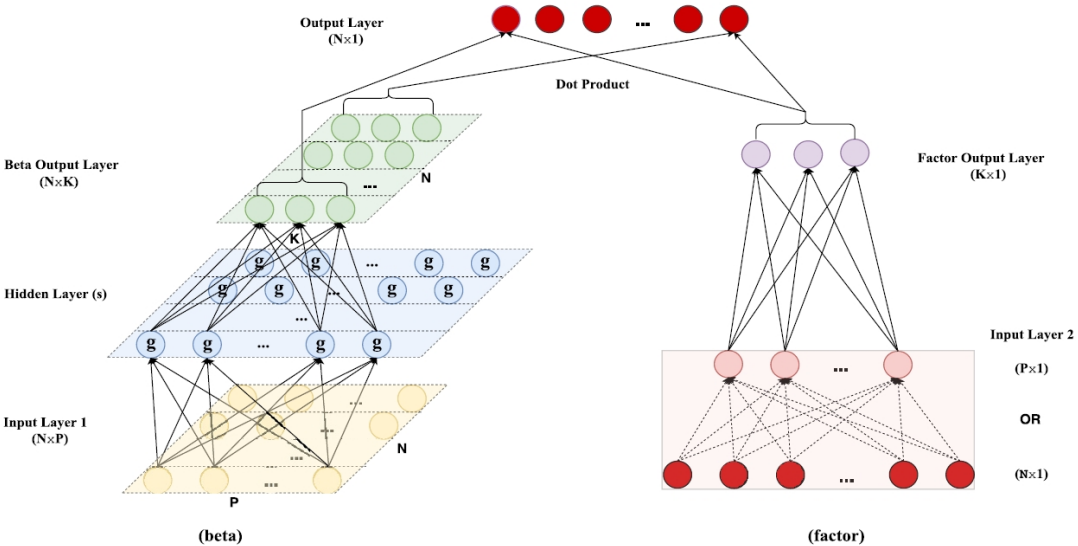
- **Real-World Financial Markets:**
  - Asset risk exposures (β) are time-varying.
  - Driven by macroeconomic conditions and firm-specific characteristics.

- **Solution:**
  - Combine Autoencoder with a *conditional* beta model.
  - Introduce *dynamic* β using firm characteristics.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

Conditional Autoencoding – Structure

4 Neural Networks



Source: Gu, Kelly and Xiu (2020)

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

- **Autoencoder (Right Side):**
  - Extract latent factors ($\boldsymbol{f}_t$) from asset returns ($\boldsymbol{r}_t$).
  - Reconstruct input data using these factors.

- **Conditional Beta Model (Left Side):**
  - Use firm-specific characteristics ($\mathbf{Z}_{t-1}$) to generate time-varying beta (β_{t-1}).
  - Capture dynamic sensitivities of assets to latent factors.

- **Output:** Combine $\boldsymbol{f}_t$ and β_{t-1} to explain asset returns:

$$\boldsymbol{r}_t = \beta_{t-1} \cdot \boldsymbol{f}_t + \boldsymbol{\epsilon}_t.$$

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models



Model Overview

- **Autoencoder (Right Side):**
  - Extract latent factors ($\boldsymbol{f}_t$) from asset returns ($\boldsymbol{r}_t$).
  - Reconstruct input data using these factors.

- **Conditional Beta Model (Left Side):**
  - Use firm-specific characteristics ($\mathbf{Z}_{t-1}$) to generate time-varying beta (β_{t-1}).
  - Capture dynamic sensitivities of assets to latent factors.

- **Output:** Combine $\boldsymbol{f}_t$ and β_{t-1} to explain asset returns:

$$\boldsymbol{r}_t = \beta_{t-1} \cdot \boldsymbol{f}_t + \boldsymbol{\epsilon}_t.$$

- **Two pairs: {PCA, autoencoder} and {IPCA, conditional autoencoder}, reflecting linear versus nonlinear activation functions, while the first (second) pair refers to unconditional (conditional) asset pricing.**

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models



Table of Contents

5 Deep Learning

- Recap and Nonlinearity
- Regression Trees
- Ensemble Models
- Neural Networks
- Deep Learning
- Economic Restrictions

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

Recall No-Arbitrage Condition

- No-arbitrage condition:

$$\mathbb{E}_t[M_{t+1}R^e_{i,t+1}] = 0,$$

for $t = 1, \cdots, T$ and $i = 1, \cdots, N$, where:

  - $R^e_{i,t+1}$ is the excess return of asset i at $t+1$.
  - $\mathbb{E}_t[.]$ is the expected value conditioning on the information set at t.
  - M_{t+1} is the SDF at $t+1$.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

Recall No-Arbitrage Condition

- No-arbitrage condition:

$$\mathbb{E}_t[M_{t+1}R^e_{i,t+1}] = 0,$$

for $t = 1, \cdots, T$ and $i = 1, \cdots, N$, where:

  - $R^e_{i,t+1}$ is the excess return of asset i at $t+1$.
  - $\mathbb{E}_t[.]$ is the expected value conditioning on the information set at t.
  - M_{t+1} is the SDF at $t+1$.

- Conditioning **down** using instruments, i.e., managed portfolios:

$$\mathbb{E}[M_{t+1}R^e_{i,t+1}g(\boldsymbol{I}_t, \boldsymbol{I}_{i,t})] = 0,$$

where $\boldsymbol{I}_t$ and $\boldsymbol{I}_{i,t}$ represents marco-economic information and firm-specific characteristics, respectively. **These managed portfolios are test assets.**

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

- Project the SDF on the return space:

$$M_{t+1} = 1 - \sum_{i=1}^{N} w_{i,t} R^e_{i,t+1},$$

- $w_{i,t}$ is a general function of the same information set $\boldsymbol{I}_t$ and $\boldsymbol{I}_{i,t}$:

$$w_{i,t} = w(\boldsymbol{I}_t, \boldsymbol{I}_{i,t}).$$

- In a traditional setting, we treat test assets (i.e., the choice of $g(.)$) as fixed and focus on estimating SDF weights $w_{i,t}$ to minimize pricing errors (a classic GMM problem).

- But, what if we take an *adversarial* approach to select $g(.)$ to challenge the SDF, i.e., the moment conditions the that lead to the largest pricing errors.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

- The problem becomes a minimax optimization problem:

$$\min_{w} \max_{g} \frac{1}{N} \sum_{j=1}^{N} \left\| \mathbb{E}\left[\left(1 - \sum_{i=1}^{N} w(\boldsymbol{I}_t, \boldsymbol{I}_{i,t}) R^e_{i,t+1} \right) R^e_{i,t+1} g(\boldsymbol{I}_t, \boldsymbol{I}_{i,t}) \right] \right\|^2 .$$

- This problem can be modeled as a **zero-sum game**, where one player, the asset pricing modeler, aims to choose an SDF ($w(.)$ part), while the adversary searches for moment conditions ($g(.)$ part) under which the SDF fails to price.

- This can be done via a generative adversarial network (GAN). This is the work of Chen, Pelger and Zhu (2024).

Note: This slide incorporates material from Professor Doron Avramov's lecture notes, used with permission.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

- There are two competing neural networks:
  1. **SDF Model**: Minimizes pricing errors of test assets.
  2. **Adversarial network**: Selects managed portfolios (test assets) to maximize pricing errors.

- Iterative process:
  1. w is updated by minimizing the loss function given g: $\hat{w} = \min_w L(w|g)$.
  2. g is updated by maximizing the loss given w: $\hat{g} = \max_g L(g|w)$.

- We first find portfolios that are the most mispriced and then tune the SDF to also price these assets.

- The process is repeated until the adversary cannot find portfolios with large enough pricing errors.

Note: This slide incorporates material from Professor Doron Avramov's lecture notes, used with permission.

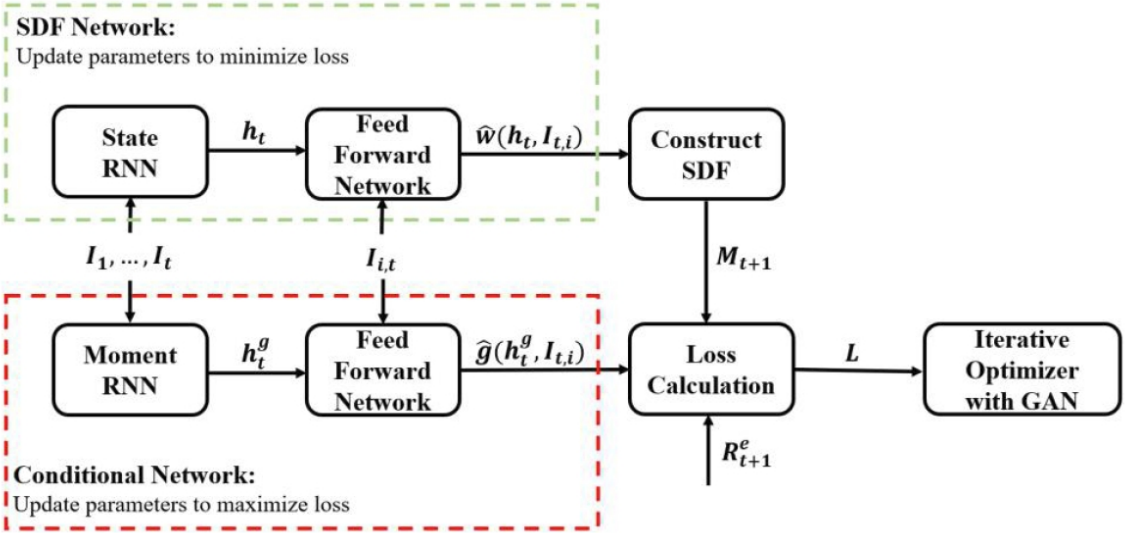
© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

GAN – An Illustration

- GAN is a **data-driven generalization** for the research protocol conducted in asset pricing in the last decades.
- To illustrate, assume that the SDF model uses the Fama-French five-factor model.
- The adversary propose momentum sorted test assets, that is g is a vector of indicator functions for different quantiles of past returns.
- The SDF model needs to revise the candidate SDF, for example, by adding a momentum factor.
- Next, the adversary searches for other mispriced anomalies or states of the economy, which the SDF model will exploit in revising the SDF.

Note: This slide incorporates material from Professor Doron Avramov's lecture notes, used with permission.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models



© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

GAN – Some Empirical Results

	SR			EV			Cross-Sectional R^2		
Model	Train	Valid	Test	Train	Valid	Test	Train	Valid	Test
LS	1.80	0.58	0.42	0.09	0.03	0.03	0.15	0.00	0.14
EN	1.37	1.15	0.50	0.12	0.05	0.04	0.17	0.02	0.19
FFN	0.45	0.42	0.44	0.11	0.04	0.04	0.14	-0.00	0.15
GAN	2.68	1.43	0.75	0.20	0.09	0.08	0.12	0.01	0.23

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models



Table of Contents

6 Economic Restrictions

- Recap and Nonlinearity
- Regression Trees
- Ensemble Models
- Neural Networks
- Deep Learning
- Economic Restrictions

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

- **Is it really out-of-sample?** These machine learning model papers used features (i.e., firm characteristics) identified by empirical asset pricing literature as anomalies (or factors) related to stock returns

- **BUT**, and this is a huge but:
  - Harvey, Liu, and Zhu (2016) examined about 300 anomalies, 27% to 53% are likely to be **false discoveries**.
  - Hou, Xue and Zhang (2020) replicated 452 anomalies, 82% turn insignificant upon **excluding microcaps + value-weighting**.
  - Anomaly profits mostly originate from **short-leg distressed stocks** and often disappear in recent years (Avramov et al. 2013).

Note: This slide incorporates material from Professor Doron Avramov's lecture notes, used with permission.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models



Validity of ML Models under Economic Restrictions

- Will the performance of machine learning models persist under economic restrictions?
  - i.e., exclude difficult-to-arbitrage stocks.
- Does the return predictability of machine learning signals vary over time?
  - i.e., exclude market states with high limits to arbitrage.
- Avramov, Cheng and Metzker (2023) addressed these issues.
  - They exclude the following from the cross-section: microcaps (market cap smaller than the 20th NYSE size percentile), no-rating firms (firms without credit rating), and distressed firms ($[-12, +12]$ months around an issuer credit rating downgrade).
  - They use the following market state variables: investor sentiment, realized market volatility, VIX, and market illiquidity. High values of these variables indicate high limits to arbitrage.

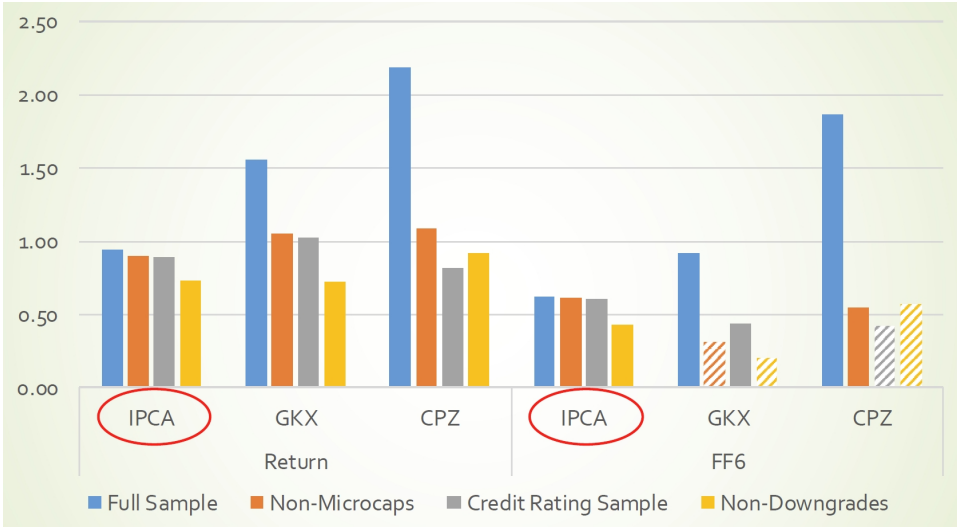
© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

- Well, you have seen this before...

Model	Full Sample	Exclude Tiny Market Cap	Exclude No Credit Rating	Exclude Financially Distressed
Panel A: Absolute Return (%)				
Gu, Kelly and Xiu (2020)	1.56	1.05	1.02	0.72
	(4.53)	(3.24)	(3.18)	(2.49)
Chen, Pelger and Zhu (2023)	2.13	1.08	0.82	0.92
	(6.37)	(4.06)	(2.83)	(2.91)
Kelly, Pruitt and Su (2019)	0.95	0.91	0.89	0.73
	(5.62)	(5.57)	(5.08)	(4.08)
Gu, Kelly and Xiu (2021)	1.16	1.11	0.87	0.67
	(4.17)	(4.22)	(2.97)	(2.22)
Panel B: Alpha (%) against FF5 + MOM				
Gu, Kelly and Xiu (2020)	0.92	0.31	0.43	0.20
	(4.08)	(1.51)	(2.05)	(0.92)
Chen, Pelger and Zhu (2023)	1.87	0.55	0.42	0.57
	(4.86)	(2.23)	(1.46)	(1.82)
Kelly, Pruitt and Su (2019)	0.62	0.61	0.61	0.43
	(3.31)	(3.71)	(3.38)	(2.37)
Gu, Kelly and Xiu (2021)	0.75	0.39	0.19	0.05
	(3.01)	(2.03)	(0.79)	(0.20)

Data excerpted from Avramov, Cheng and Metzker (2023).

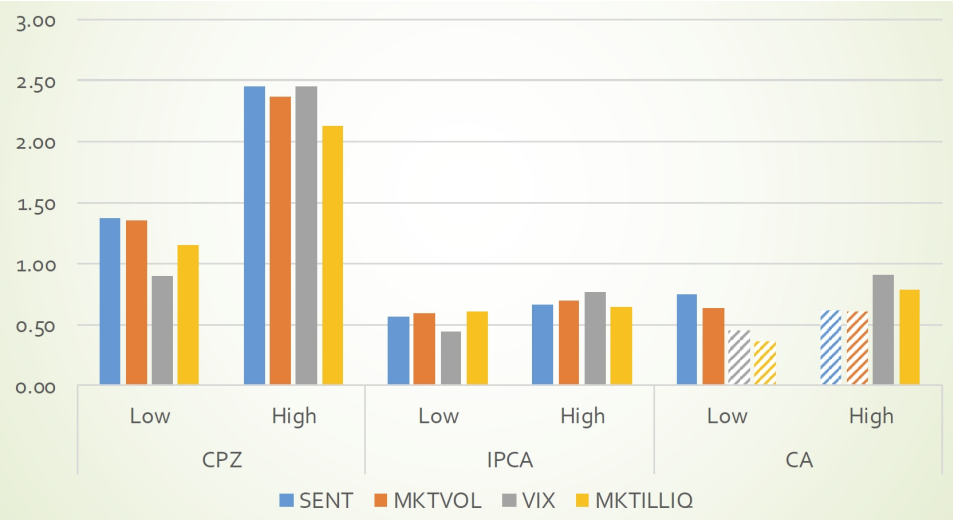
© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models



Source: Professor Doron Avramov's lecture note.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models

Time-Varying Predictability (Full Sample)



Source: Professor Doron Avramov's lecture note.

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models



Advanced Machine Learning Models

Questions?

© 2025 – 2026 Chuan Shi | Advanced Machine Learning Models